Swinburne University of Technology

Faculty of Science, Engineering and Technology

ASSIGNMENT COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 4, Binary Search Trees & In-Order Traversal
Due date: Apr 12, 23:59
Lecturer: Dr. Bui Van Vuong

Your name: Nguyen Duc Manh **Your student id:** 105547489 / SWH02701

Check Tutorial	Mon 10:30	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Tues 16:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30

Marker's comments:

Problem	Marks	Obtained
1	94	
2	42	
3	8+86=94	
Total	230	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener:_____

```
1
2  // COS30008, Problem Set 4, Problem 1, 2022
3
4  #pragma once
5
6  #include <stdexcept>
7  #include <algorithm>
8
9  template<typename T>
10 struct BinaryTreeNode
11 {
12     using BNode = BinaryTreeNode<T>;
13     using BTreeNode = BNode*;
14
15     T key;
16     BTreeNode left;
17     BTreeNode right;
18
19     static BNode NIL;
20
21     const T& findMax() const
22     {
23         if ( empty() )
24         {
25             throw std::domain_error( "Empty tree encountered." );
26         }
27
28         return right->empty() ? key : right->findMax();
29     }
30
31     const T& findMin() const
32     {
33         if ( empty() )
34         {
35             throw std::domain_error( "Empty tree encountered." );
36         }
37
38         return left->empty() ? key : left->findMin();
39     }
40
41     bool remove( const T& aKey, BTreeNode aParent )
42     {
43         BTreeNode x = this;
44         BTreeNode y = aParent;
45
46         while ( !x->empty() )
47         {
48             if ( aKey == x->key )
49             {
```

```
                break;
            }

            y = x;                                      // new parent

            x = aKey < x->key ? x->left : x->right;
        }

        if ( x->empty() )
        {
            return false;                               // delete failed
        }

        if ( !x->left->empty() )
        {
            const T& lKey = x->left->findMax();         // find max to left
            x->key = lKey;
            x->left->remove( lKey, x );
        }
        else
        {
            if ( !x->right->empty() )
            {
                const T& lKey = x->right->findMin();    // find min to right
                x->key = lKey;
                x->right->remove( lKey, x );
            }
            else
            {
                if ( y != &NIL )                        // y can be NIL
                {
                    if ( y->left == x )
                    {
                        y->left = &NIL;
                    }
                    else
                    {
                        y->right = &NIL;
                    }
                }

                delete x;                               // free deleted node
            }
        }

        return true;
```

```
    }

    // PS4 starts here

    BinaryTreeNode() : left(this), right(this) {};
    BinaryTreeNode( const T& aKey ) :key(aKey), left(&NIL), right(&NIL) {};
    BinaryTreeNode( T&& aKey ) : key(std::move(aKey)), left(&NIL), right(&NIL) {};

    ~BinaryTreeNode() {
        if (left != &NIL) {
            delete left;
        }
        if (right != &NIL) {
            delete right;
        }
    };

    bool empty() const { return this == &NIL; };
    bool leaf() const { return (left == &NIL) && (right == &NIL); };
    size_t height() const {
        if (empty())
            throw std::domain_error("Empty tree encountered");

        size_t l = left->empty() ? 0 : left->height() + 1;
        size_t r = right->empty() ? 0 : right->height() + 1;

        return std::max(l, r);
    }

    bool insert(const T& aKey)
    {
        if (aKey == key) return false;

        if (aKey < key)
        {
            if (left == &NIL)
            {
                left = new BNode(aKey);
                return true;
            }
            return left->insert(aKey);
        }
        else
        {
            if (right == &NIL)
            {
                right = new BNode(aKey);
```

```
                return true;
            }
            return right->insert(aKey);
        }
    }
};

template<typename T>
BinaryTreeNode<T> BinaryTreeNode<T>::NIL;

```

```

// COS30008, Problem Set 4, Problem 2, 2022

#pragma once

#include "BinaryTreeNode.h"

#include <stdexcept>

// Problem 3 requirement
template<typename T>
class BinarySearchTreeIterator;

template<typename T>
class BinarySearchTree
{
private:

    using BNode = BinaryTreeNode<T>;
    using BTreeNode = BNode*;

    BTreeNode fRoot;

public:

    BinarySearchTree() : fRoot(&BNode::NIL) {};

    ~BinarySearchTree() {
        if (!empty()) {
            delete fRoot;
        }
    };

    bool empty() const { return fRoot == &BNode::NIL; };
    size_t height() const {
        if (empty())
            throw std::domain_error("Empty tree has no height.");
        return fRoot->height();
    };

    bool insert(const T& aKey) {
        if (empty()) {
            fRoot = new BNode(aKey);
            return true;
        }
        return fRoot->insert(aKey);
    };

    bool remove(const T& aKey) {
```

```
        if (empty()) return false;

        // if the root is a leaf and matches the key,
        // => update fRoot to NIL after deleting it.
        if (fRoot->key == aKey && fRoot->leaf()) {
            delete fRoot;
            fRoot = &BNode::NIL;
            return true;
        }

        return fRoot->remove(aKey, &BNode::NIL);
    };

    // Problem 3 methods

    using Iterator = BinarySearchTreeIterator<T>;

    // Allow iterator to access private member variables
    friend class BinarySearchTreeIterator<T>;

    Iterator begin() const {
        return Iterator(*this);
    }

    Iterator end() const {
        return Iterator(*this).end();
    };
};

```

```cpp

// COS30008, Problem Set 4, Problem 3, 2022

#pragma once

#include "BinarySearchTree.h"

#include <stack>

template<typename T>
class BinarySearchTreeIterator
{
private:

    using BSTree = BinarySearchTree<T>;
    using BNode = BinaryTreeNode<T>;
    using BTreeNode = BNode*;
    using BTNStack = std::stack<BTreeNode>;

    const BSTree& fBSTree;      // binary search tree
    BTNStack fStack;            // DFS traversal stack

    void pushLeft(BTreeNode aNode) {
        while (aNode != &BNode::NIL) {
            fStack.push(aNode);
            aNode = aNode->left; // move to the left node
        }
    };

public:

    using Iterator = BinarySearchTreeIterator<T>;

    BinarySearchTreeIterator(const BSTree& aBSTree) : fBSTree(aBSTree) {
        pushLeft(fBSTree.fRoot);
    };

    const T& operator*() const { return fStack.top()->key; };

    Iterator& operator++() {
        BTreeNode lNode = fStack.top();
        fStack.pop();
        if (lNode->right != &BinaryTreeNode<T>::NIL)
        {
            pushLeft(lNode->right);
        }
        return *this;
    };

```

```
    Iterator operator++(int) {
        Iterator temp = *this;
        ++(*this);
        return temp;
    };

    bool operator==(const Iterator& aOtherIter) const {
        return
            (&fBSTree == &aOtherIter.fBSTree) &&
            (fStack == aOtherIter.fStack);
    };

    bool operator!=(const Iterator& aOtherIter) const {
        return !(*this == aOtherIter);
    };

    Iterator begin() const {
        Iterator temp = *this;
        temp.fStack = BTNStack();
        temp.pushLeft(temp.fBSTree.fRoot);
        return temp;
    };

    Iterator end() const {
        Iterator temp = *this;
        temp.fStack = BTNStack();
        return temp;
    }
};

```